BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Coverage

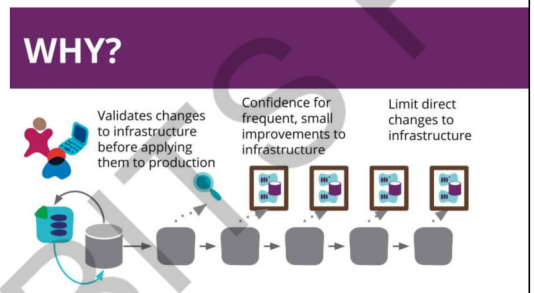
- Managing Infrastructure and Environments (Production, pre-production, Test, Developer Environment)
- Environment provisioning.
- Automating and Managing Server Provisioning
- Enterprise solutions Chef, Puppet and Ansible
- Managing on-demand infrastructure, Auto scaling

Introduction

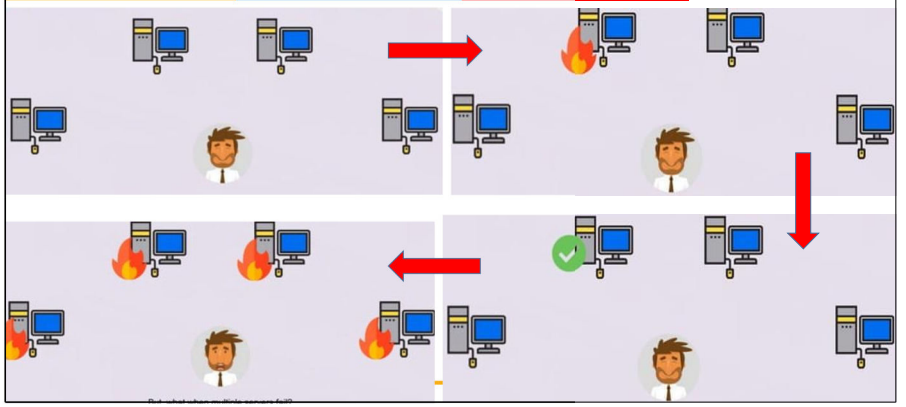
- Configuration management is a systems engineering process for establishing consistency of a product's attributes throughout its life.
- In the technology world, configuration management is an IT management process that tracks individual configuration items of an IT system.
- IT systems are composed of IT assets that vary in granularity. An IT asset may represent a piece of software, or a server, or a cluster of servers.
- Software configuration management is a systems engineering process that tracks and monitors changes to a software systems configuration metadata.
- Configuration management is commonly used alongside version control and CI/CD infrastructure.

Why is Configuration Mngt important?

- Configuration management helps engineering teams build robust and stable systems through the use of tools that automatically manage and monitor updates to configuration data.



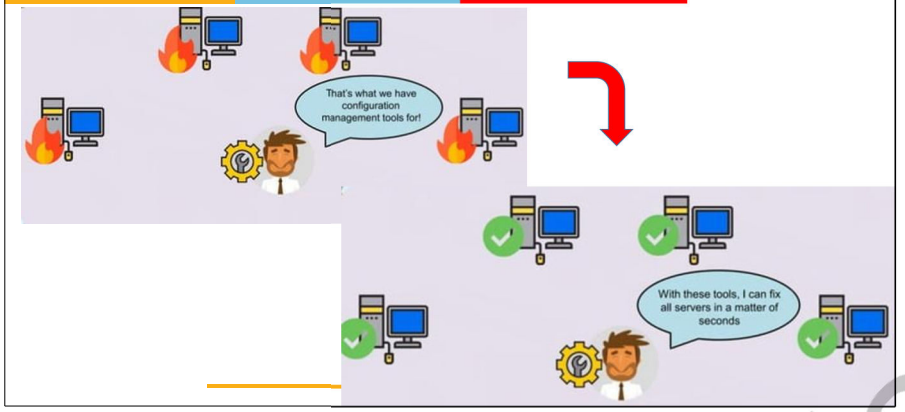
Why is Configuration Mngt important?



innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Why is Configuration Mngt important?



How configuration management fits with DevOps, CI/CD

- The rise of cloud infrastructures has led to the development and adoption of new patterns of infrastructure management.
- Complex, cloud-based system architectures are managed and deployed through the use of configuration data files.
- These new cloud platforms allow teams to specify the hardware resources and network connections they need provisioned through human and machine readable data files like YAML.
- The data files are then read and the infrastructure is provisioned in the cloud. This pattern is called Infrastructure as Code (IaC)

DevOps configuration management

- DevOps configuration is the evolution and automation of the systems administration role, bringing automation to infrastructure management and deployment.
- DevOps configuration also brings system administration responsibility under the umbrella of software engineering

CI/CD configuration management

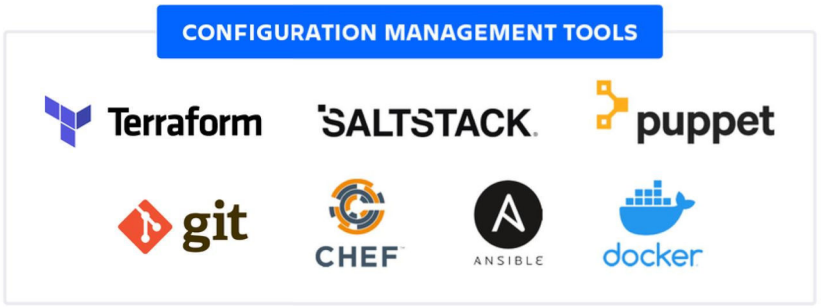
- CI/CD configuration management utilizes pull request-based code review workflows to automate deployment of code changes to a live software system.
- This same flow can be applied to configuration changes. CI/CD can be set up so that approved configuration change requests can immediately be deployed to a running system.

Elements of DevOps Configuration Management

- **Configuration Identification**: Identity the configuration of the environment to be maintained. One can also use discovery tools to identify configurations automatically.
- **Configuration Control**: Remember that it might not remain unchanged once the configuration has been identified. Consequently, there needs to be some mechanism in place to track and control changes to the configuration. Most configuration management frameworks have a change management process regulating these configurational changes.
- **Configuration Audit**: Even with control mechanisms, changes may bypass them. Configuration audits at regular intervals prevent such incidents. When choosing DevOps Configuration Management tools, select one that facilitates these three functions with the most efficiency and ease.

innovate achieve lead

Configuration management tools



What should successful DevOps Configuration Management deliver?

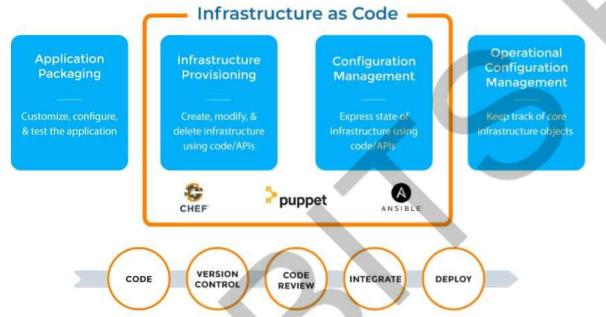
Two of the most prominent Configuration Management techniques are –

- Infrastructure-as-a-code
- Configuration-as-a-code

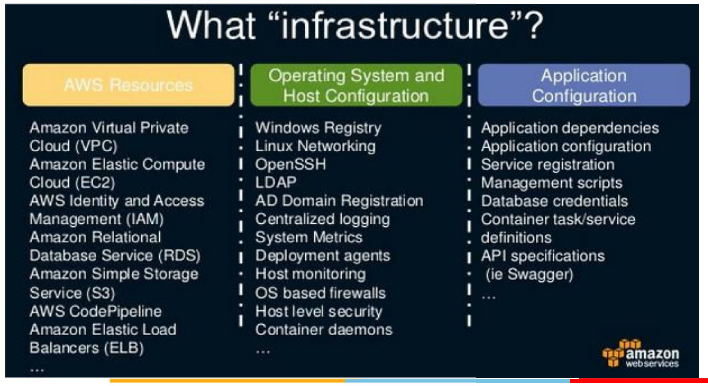
Infrastructure-as-a-Code

- Refers to the existence of code that automatically prepares the necessary environment so that it is ready for development and testing activities.
- "Environment" refers to all the resources required for DevOps operations – servers, networks, and everything comprising the IT infrastructure.
- These details are crafted as a code rather than some formal document.
- This code, pushed to the version control system

Infrastructure-as-a-Code



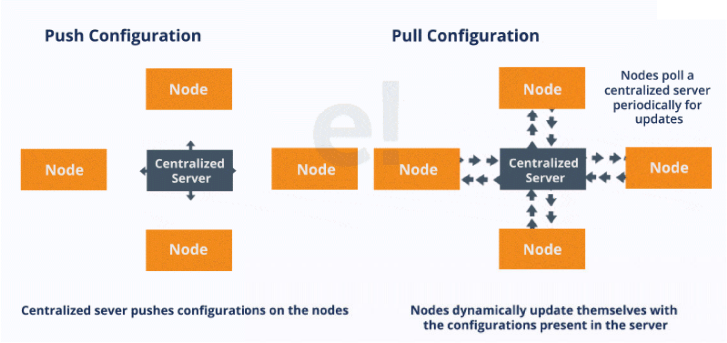
Infrastructure-as-a-Code



innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Infrastructure-as-a-Code



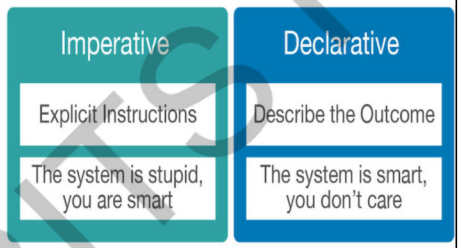
Infrastructure-as-a-Code

Tool Name	Released by	Method	Approach
Ansible Tower	Ansible	Push	Declarative & Imperative
CFEngine	CFEngine	Pull	Declarative
Chef	Chef	Pull	Imperative
Otter	Inedo	Push	Declarative & Imperative
Puppet	Puppet	Pull	Declarative
SaltStack	SaltStack	Push	Declarative & Imperative

Infrastructure-as-a-Code

Imperative vs Declarative

- Functional
- Do this, then this
- Chef
- Salt
- Ansible
- Object Oriented
- Do this and this, as you see fit
- Puppet
- Salt



What should successful DevOps Configuration Management deliver?

Configuration-as-a-Code

- Configuration-as-a-code (CaaC), like IaaC, defines the configuration of servers or any computing resources.
- Again, like IaaC, this code is pushed to a version control system as part of the software deployment pipeline.
- This automatically sets up the configuration of the relevant infrastructure so that it is ready to develop and test the software in question.
- To define configuration, get the parameters to establish the settings that will allow the software to run as expected.

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Configuration-as-a-Code

innovate achieve lead

A Real Scenario

Using a tool like Ansible to configure a web server is an example of presenting configuration as code. Conversely, you can use an Ansible script instead of manually setting up the web server. This script will automatically configure the webserver to run the correct software version, set up the number of worker processes, and specify its root directory – essentially creating a recipe for setting up the web server that can be saved, reviewed, and tested before running.

Benefits of Configuration Management

innovate achieve lead

- It **reduces the risk** of unpredictable system failures and data breaches because it offers perfect visibility and tracks every change made to test environments.
- By offering detailed knowledge of all configurational elements, it **reduces costs** by lowering the possibility of duplicating technological assets.
- Offers **greater agility and better resolution** of issues by making it easy for personnel to view changes (unforeseen or otherwise) that may have led to said problems.
- Implement **faster restoration of services**. This is because the configuration, including all changes, is **automated** and **documented**. Not only is it easier to detect the problem, but it is easier to revert the failing environment to its last functional stage.
- Offers greater control over relevant workflows by **establishing and enforcing formalized policies** and procedures for status monitoring, asset detection, audition, change implementation, etc.

Benefits of Configuration Management

innovate achieve lead

A Real Scenario

Assume you wish to build a web application on a cloud provider such as AWS. Traditionally, you would have to establish a virtual server manually, install the operating system, configure the network settings, and install the application-specific software.

Solution:

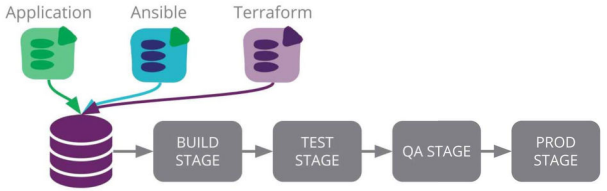
With Infrastructure as Code, you'd write a script that outlines the infrastructure required to execute your application. This script will tell you what kind of server you need, how much memory it should have, and what software needs to be installed. After you've built the script, you can use a tool like Terraform to create the server, configure it, and install the required software with a single command

Usage of Config Management tools

innovate achieve lead

SIMPLE PIPELINE DESIGN

Deploy application, configuration, and infrastructure

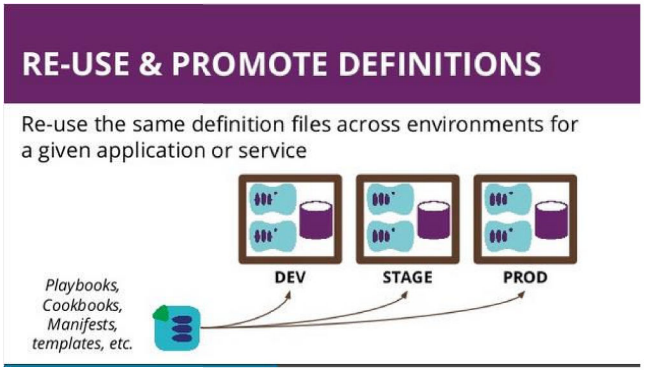


innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Usage of Config Management tools

innovate achieve lead



Usage of Config Management tools

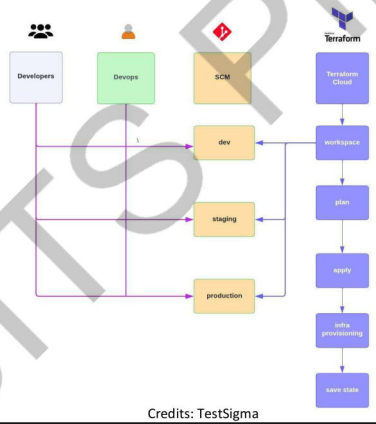
innovate achieve lead

- It **reduces the risk** of unpredictable system failures and data breaches because it offers perfect visibility and tracks every change made to test environments.
- By offering detailed knowledge of all configurational elements, it **reduces costs** by lowering the possibility of duplicating technological assets.
- Offers **greater agility and better resolution** of issues by making it easy for personnel to view changes (unforeseen or otherwise) that may have led to said problems.
- Implement **faster restoration of services**. This is because the configuration, including all changes, is **automated** and **documented**. Not only is it easier to detect the problem, but it is easier to revert the failing environment to its last functional stage.
- Offers greater control over relevant workflows by **establishing and enforcing formalized policies** and procedures for status monitoring, asset detection, audition, change implementation, etc.

Usage of Config Management tools

innovate achieve lead

- First, you make changes to the terraform git repository.
- Then Terraform Cloud picks up the changes from the git repo(SCM), where the terraform plan is applied and approved.
- Finally, once the plan is approved, the AWS infrastructure gets created!
- Pushing code to a Github repository is considered Continuous Integration (CI). Terraform cloud pulls the code from the Github repository and provisions it to AWS, representing Continuous Deployment (CD).

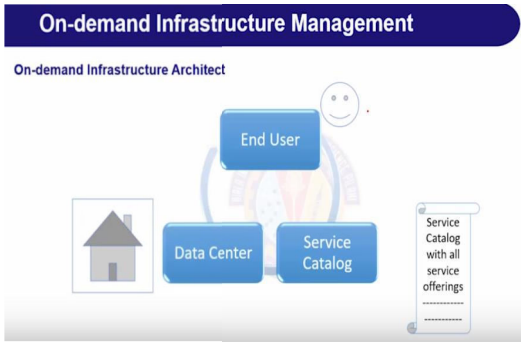


Credits: TestSigma

On-Demand Infrastructure Management

innovate achieve lead

- New IaaS technology is making it for on-demand infrastructure.
- IaaS option provides users with on-demand access.
- With IaaS, small businesses no longer have to build infrastructure for data storage internally.
- Can pay for what they need as they grow.



innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

On-Demand Infrastructure Management

innovate achieve lead

On-demand Infrastructure Management

Service Providers

- IaaS Providers:
  - bmc
  - DellEMC
  - Dropbox
  - Cisco
  - Juniper
  - Vmware
- PaaS Providers:
  - Redhat
  - Azure
  - at & t
  - Google
  - Verizon
- SaaS Providers:
  - Salesfroce
  - Netapp
  - Coupa

On-Demand Infrastructure Management

innovate achieve lead

Benefits

- Cost Savings
- Scalability and flexibility
- Faster time to market
- Support for DR and high availability
- Focus on business growth

Modeling and managing infrastructure

innovate achieve lead

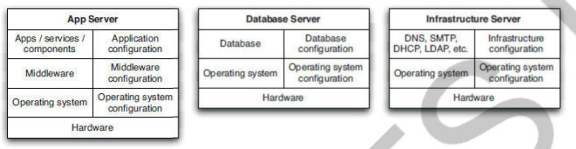


Figure 11.1 *Types of servers and their configuration*

- Controlling access to prevent anyone from making a change without approval
- Defining an automated process for making changes to your infrastructure
- Monitoring your infrastructure to detect any issues as soon as they occur

Managing Infrastructure

innovate achieve lead

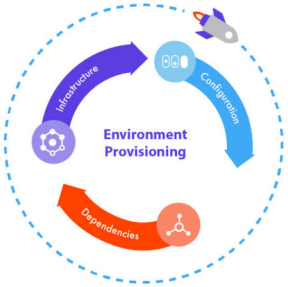
- The desired state of your infrastructure should be specified through version-controlled configuration.
- Infrastructure should be autonomic—that is, it should correct itself to the desired state automatically.
- You should always know the actual state of your infrastructure through instrumentation and monitoring.

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Environment provisioning

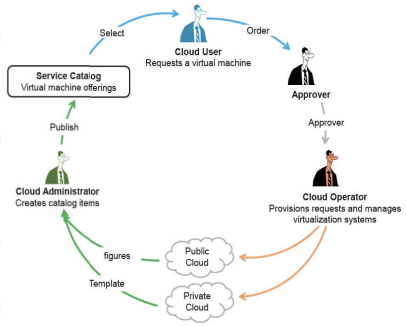
- Environment provisioning is a key part of a continuous delivery process.
- The idea is simple: we should not only build, test and deploy application code, but also the underlying application environment.
- The application environment consists in 3 main areas:
  - Infrastructure
  - Configuration
  - Dependencies



Environment provisioning

During provisioning, the following elements are selected:

- Software platform and development frameworks for ready-made environments.
- Common frontend or backend instances.
- High-availability options (failover, IT resource clustering).
- Monitors SLA metrics.
- Basic software configurations (operating system, pre-installed software for new virtual servers).
- IaaS resource instances from a number of available hardware-related configurations and options



Environment provisioning – Use case

- *Onboarding* new application teams have to deal with organizational complexity and adhere to standards and release policies. Provisioning a baseline environment catalog will help circumvent organizational obstacles and technological challenges.
- *Coordinating* code releasing and provisioning to make sure infrastructure changes arrive just-in-time with the code that requires it.
- *Dev and QA environment provisioning*, to simplify and control the process of procuring and automating environment generation.
- *Self-service provisioning*, offering users a "catalog" with a set of service requests and tasks that can be launched and managed.
- *Infrastructure code* is being pushed and continuously deployed by developers (ie. Dockerfiles or Chef recipes), but require more control and end-to-end visibility.

Automated Server Provisioning

- Automated provisioning works by configuring permissions and resource access within an IAM platform based on predefined settings.
- The organization would create automation rules that automatically give new users certain resource access rights based on their role, group, and company policies

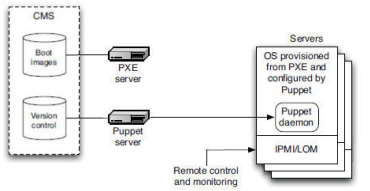


Figure 11.2 *Automated provisioning and configuration of servers*

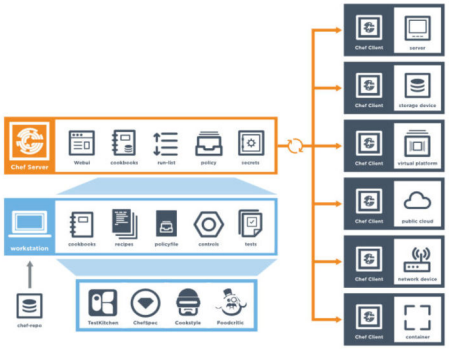
innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Chef

- Chef Server
- Workstation
- Chef-Repo
- Chef Client
- Chef Recipe

```
cookbook_file "/etc/resolv.conf" do
  action :create
  owner "root"
  group "root"
  mode "0644"
end
```



Introducing Chef: Automating Your Infrastructure

Client-server architecture:
- Manages nodes using a central Chef Server

Cookbooks:
- Reusable configuration recipes written in Ruby DSL

Benefits:
- Streamlined infrastructure provisioning & configuration
- Improved consistency & reduced errors
- Scalability for large deployments

Understanding Chef Architecture

Chef Server:
- Central hub that stores cookbooks, manages clients, and generates configuration profiles (cookbook recipes)

Chef Workstation:
- Development environment for creating and testing cookbooks

Chef-Client:
- Lightweight agent installed on managed nodes
- Periodically checks in with Chef Server for updates
- Applies configuration changes based on received recipes

Chef in Action: Automating Web Server Configuration

Example:
- Configuring a web server with Chef

Cookbook:
- Defines steps to install Apache, configure ports, set document root

Chef Server:
- Generates configuration profile with these recipes

Chef-Client:
- Applies the profile, installing Apache and configuring it according to the cookbook

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Puppet: Automating IT with Ease
Open-source IT automation tool

innovate achieve lead

Agent-based architecture:
- Relies on lightweight agents installed on managed nodes

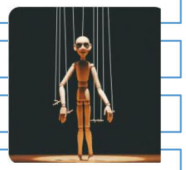
Manifests:
- Configuration files written in a declarative language (Puppet Language)

Example:
- Managing Apache web server

Manifest:
- Defines desired state (e.g., install package httpd, configure port 80)

Puppet Agent:
- Installs/configures Apache to match the desired state

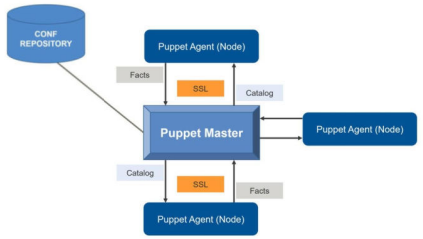
Benefits:
- Consistent configuration across systems
- Reduced manual errors
- Improved infrastructure management



Puppet

innovate achieve lead

- Puppet is:
  - a declarative language for expressing system configuration.
  - a client and server for distributing it.
  - a library for realizing the configuration.
- Puppet is an abstraction layer between the system administration and the system.
- Puppet is written in Ruby and distributed under the GPL.



Puppet Architecture

innovate achieve lead

Puppet Master:
- Central server that stores manifests, manages agents, and compiles catalogs

Puppet Agent:
- Lightweight agent installed on managed nodes
- Periodically polls the Puppet Master for configurations
- Applies configuration changes based on received catalogs

Catalogs:
- Translated versions of manifests, specific to each agent

Puppet – How it Works?

innovate achieve lead

- Client (Agent) - puppetd
- Server - puppetmasterd

- The client connects to the server every half an hour.
- The server compiles the configuration for the client (based on what you write) and sends it to the client.
- The client checks the configuration it receives (or a cached version if it can't contact the server) against what is really on the machine and tries to fix whatever is wrong.
- Client (node) configuration can be stored in LDAP, in a database or in the puppet configuration files.

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Benefits of Puppet

innovate achieve lead

Scalability:
- Manages large and complex IT infrastructures efficiently

Security:
- Enforces configuration policies for improved security posture

Compliance:
- Helps meet regulatory compliance requirements

Version Control:
- Tracks configuration changes through version control integration

Auditing:
- Provides detailed audit logs for troubleshooting and analysis

Ansible

innovate achieve lead

- Ansible is a radically simple IT automation platform that makes your applications and systems easier to deploy.
- It support configuration management with examples as below.
  - Configuration of servers
  - Application deployment
  - Continuous testing of already install application
  - Provisioning
  - Orchestration
  - Automation of tasks

ANSIBLE

Ansible: Open-source IT automation tool

innovate achieve lead

Enables infrastructure provisioning, configuration management, and application deployment

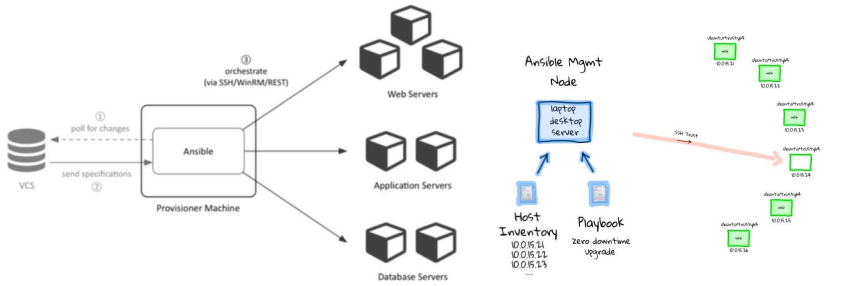
Agentless architecture:
- Manages systems using SSH or PowerShell remoting

Playbooks:
- Define automation tasks in human-readable YAML files

Modules:
- Reusable code components for specific tasks

Ansible – Agentless Architecture

innovate achieve lead



innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

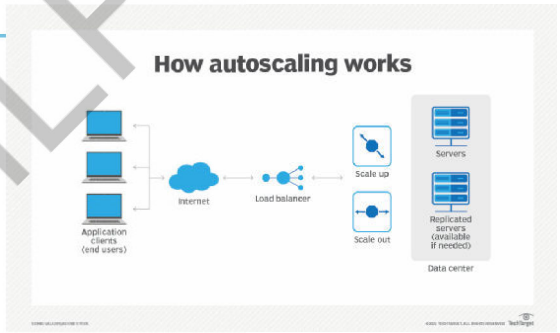
Autoscaling

- Autoscaling is a cloud computing feature that enables organizations to scale cloud services such as server capacities or virtual machines up or down automatically, based on defined situations such as traffic or utilization levels.
- The overall benefit of autoscaling is that it eliminates the need to respond manually in real-time to traffic spikes that merit new resources and instances by automatically changing the active number of servers.
- Each of these servers requires configuration, monitoring and decommissioning, which is the core of autoscaling.

Autoscaling

Advantages:

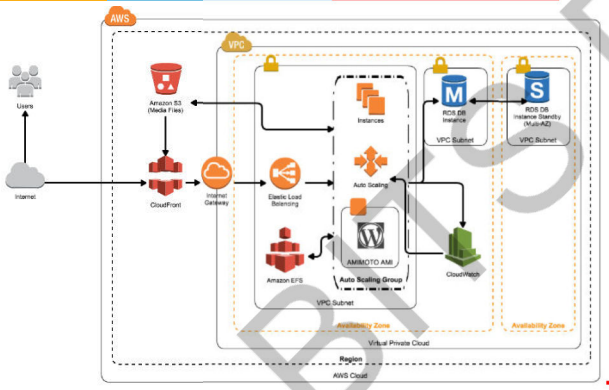
- Cost
- Security
- Availability



Types-

- Predictive Autoscaling vs Scheduled Autoscaling
- Horizontal Auto Scaling vs Vertical Auto Scaling

Autoscaling - AWS

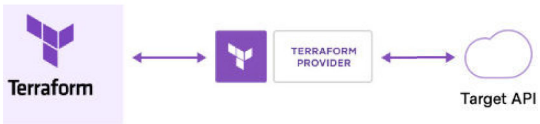


Terraform

TERRAFORM

Terraform is a tool for building, changing, and versioning infrastructure safely and efficiently.

Terraform is a tool for building, changing, and versioning infrastructure safely and efficiently. Terraform can manage existing and popular service providers as well as custom in-house solutions.



innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Terraform

innovate achieve lead

TERRAFORM FACTS

Latest version : 0.7.x

Open-source

Active development:

▼ GitHub Issues (5 - 15 issues resolving daily)

▼ Growing community (IRC, Mailing list, Stack Overflow)

▼ CHANGELOG.md

Terraform – Getting started

innovate achieve lead

Installing Terraform

To install Terraform, find the appropriate package for your system and download it. Terraform is packaged as a zip archive.

Example for Linux/Mac - Type the following into your terminal:

```
PATH=/usr/local/terraform/bin:/home/your-user-name/terraform:$PATH
```


Terraform – Commands

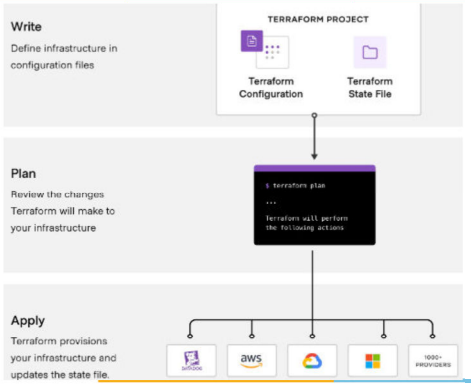
innovate achieve lead

```
$ terraform
usage: terraform [--version] [--help] <command> [<args>]

Available commands are:
    apply       Builds or changes infrastructure
    destroy     Destroy Terraform-managed infrastructure
    get         Download and install modules for the configuration
    graph       Create a visual graph of Terraform resources
    init        Initializes Terraform configuration from a module
    output      Read an output from a state file
    plan        Generate and show an execution plan
    push        Upload this Terraform module to Atlas to run
    refresh     Update local state file against real resources
    remote      Configure remote state storage
    show        Inspect Terraform state or plan
    taint       Manually mark a resource for recreation
    validate    Validates the Terraform files
    version     Prints the Terraform version
```


Terraform

innovate achieve lead



Credits: Terraform by Hashicorp

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Terraform Commands

Terraform is a powerful Infrastructure as Code (IaC) tool.

It allows you to define, provision, and manage infrastructure across various cloud platforms and on-premises environments.

By understanding these commands, you can streamline your infrastructure automation process.

Terraform init

- Initializes the Terraform workspace
- Downloads required plugins and modules
- Sets up the backend configuration for storing Terraform state

first step whenever you start working on a new Terraform project or clone an existing one from version control.

It prepares your working directory by downloading the necessary provider plugins and modules based on the configurations in your Terraform code.

These plugins act as bridges between Terraform and specific cloud platforms or infrastructure providers (e.g., AWS, Azure, GCP).

Modules are reusable configuration components that can be shared and incorporated into your Terraform code for modularity and code organization.

Additionally, terraform init sets up the backend configuration, which specifies where Terraform stores the state of your infrastructure resources.

This state essentially acts as a record of the infrastructure you've managed with Terraform.

Terraform plan

- Creates an execution plan for infrastructure changes
- Analyzes the current state and desired state
- Shows additions, modifications, and deletions

The terraform plan command is a crucial step before applying any infrastructure changes defined in your Terraform configuration files.

It analyzes the current state of your infrastructure (if any) and compares it to the desired state you've specified in your Terraform code.

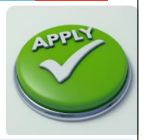
Based on this comparison, terraform plan generates an execution plan that outlines the actions Terraform will take to achieve the desired state.

This plan includes details on resources that will be created, modified, or destroyed.

It's vital to review this plan carefully before proceeding with the actual changes.

By doing so, you can identify any potential issues or unexpected modifications before they impact your production environment.

Terraform apply



- Applies the planned infrastructure changes
- Creates, modifies, or destroys resources based on the plan
- Prompts for confirmation before execution

The terraform apply command is the final step in the workflow where Terraform executes the actions outlined in the previously generated plan.

It creates, modifies, or destroys infrastructure resources based on the desired state defined in your Terraform configuration.

However, before executing these changes, terraform apply prompts you for confirmation to ensure you're ready to proceed.

This is a critical safeguard to prevent accidental modifications to your infrastructure.

Once you confirm, Terraform applies the changes, and your infrastructure will be provisioned or modified according to your Terraform code.

innovate achieve lead

init, plan, and apply

these three commands (init, plan, and apply) form the foundation of working with Terraform.

By effectively utilizing these commands, you can streamline your infrastructure automation process, ensuring a clear understanding of the planned changes before applying them to your production environment.

Remember, always review the plan thoroughly before applying it, and leverage Terraform's powerful IaC capabilities to manage your infrastructure efficiently.

Enterprise solutions

- Choose your toolset.
  - Popular choices include Chef, Ansible, and Puppet
  - Choose a tool that is a good fit for your needs
  - Some considerations before making a choice
    - Infrastructure Complexity
    - Learning Curve
    - Cost
    - Advanced Tooling
    - Community and Support

Enterprise solutions

Enterprise solutions - Comparison

	Ansible	Puppet	Chef
Script Language	YAML	Custom DSL based on Ruby	Ruby
Infrastructure	Controller machine applies configuration on nodes via SSH	Puppet Master synchronizes configuration on Puppet Nodes	Chef Workstations push configuration to Chef Server, from which the Chef Nodes will be updated
Requires specialized software for nodes	No	Yes	Yes
Provides centralized point of control	No. Any computer can be a controller	Yes, via Puppet Master	Yes, via Chef Server
Script Terminology	Playbook / Roles	Manifests / Modules	Recipes / Cookbooks
Task Execution Order	Sequential	Non-Sequential	Sequential

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Enterprise solutions - Comparison

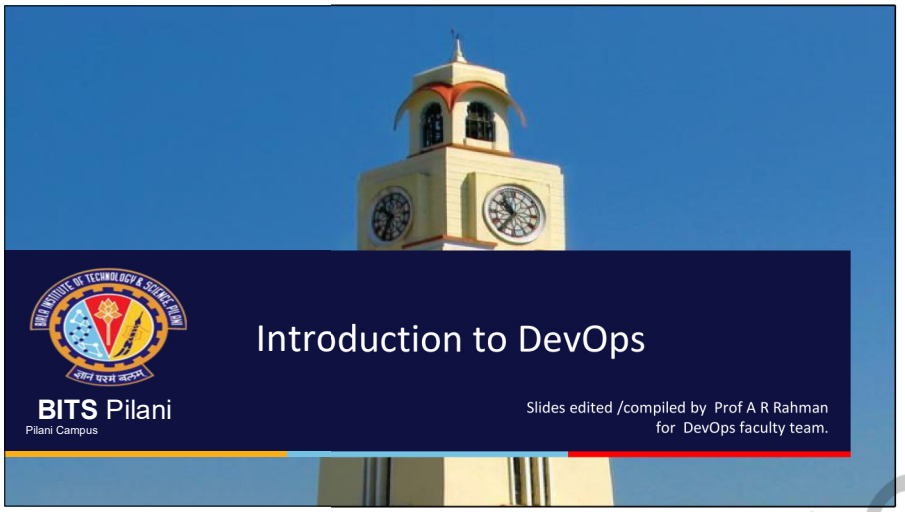
	Chef	Puppet	Ansible	SaltStack
Architecture	Server/ Client	Server/ Client	Client only	Server/ Client
Ease of setup	Moderate	Moderate	Very easy	Moderate
Language	Procedural: specify how to do a task	Declarative: specify only what to do	Procedural: specify how to do a task	Declarative: specify only what to do
Scalability	Scalable	Scalable	Scalable	Scalable
Management	Tough as it requires one to learn Ruby DSL	Tough as it requires one to learn Puppet DSL	Very easy	Very easy
Interoperability	High	High	High	High
Cloud availability	Amazon	Amazon, Azure	none	none
Communication	Knife tool	SSL	SSH	SSH

IMP Note to Self

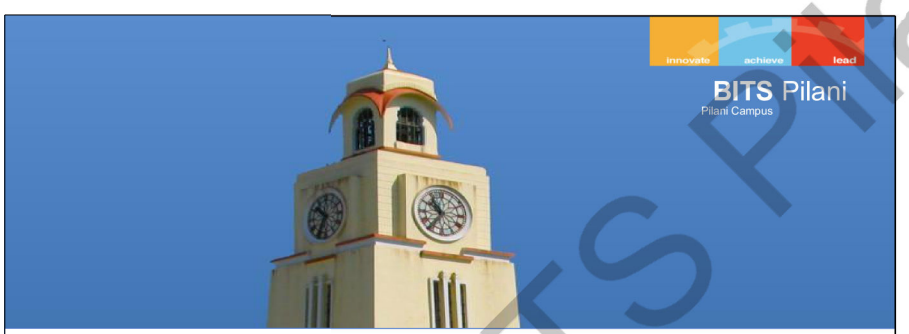
Thank You!

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES



IMP Note to Self



CSIW ZG515, Introduction to DevOps
CS 13 : DevOps – Virtualization and Containerization

IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

innovate achieve lead

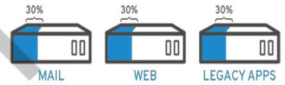
BITS Pilani
Pilani Campus

Concepts of Virtualization, Hypervisor and Virtual Machines

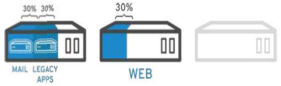
Virtualization - Definition

- Virtualization is a process that allows for more **efficient utilization of physical computer hardware** and is the foundation of cloud computing. (IBM)
- Virtualization is technology that lets users create useful IT services using resources that are **traditionally bound to hardware**. (RedHat)
- It allows users to use a **physical machine's full capacity** by distributing its capabilities among many users or environments.

Before Virtualization



After Virtualization



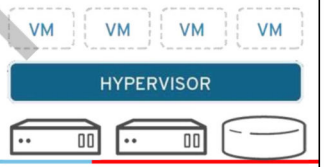
Virtualization

Vmware defines virtualization as "process of creating a software-based, or virtual, representation of something, such as applications, servers, storage or networks"

A hypervisor is a form of virtualization software used in Cloud hosting to divide and allocate the resources on various pieces of hardware

Virtual Machine is a compute resource that uses software to run programs

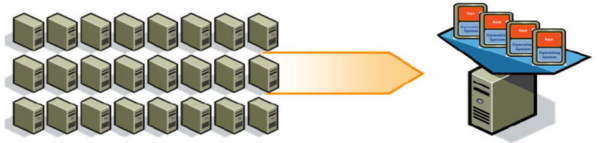
Ex: Amazon ec2



Example of Impact of Virtualization

Example of the Impact of Virtualization

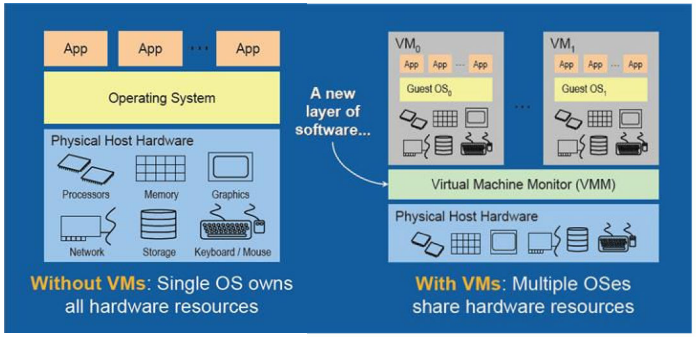
	Before	After
Servers	> 1,000	> 50
Storage	> Direct attach	> Tiered SAN and NAS
Network	> 3000 cables/ports	> 300 cables/ports
Facilities	> 200 racks	> 10 racks
	> 400 power whips	> 20 power whips



innovate achieve lead

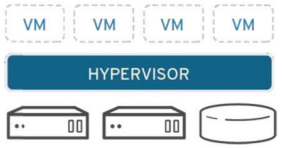
BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Sneak Peek into Virtualization



Hypervisor

- A hypervisor is the software layer that coordinates VMs.
- Interface between Virtual Machine and physical hardware. All applications running on the virtualized machine, run as if they are on their own dedicated machine, where the operating system, libraries, and other programs are dedicated to the guest and unconnected to the host OS, sitting below it.
- Virtual Machine isolation
- Installation
  - Directly on Hardware
  - On a operating system



Examples - KVM (Kernel Based Virtual Machine), Xen, Hyper-V, ESXi, Virtual Box, Parallels for Mac

Types of Hypervisor

Type 1

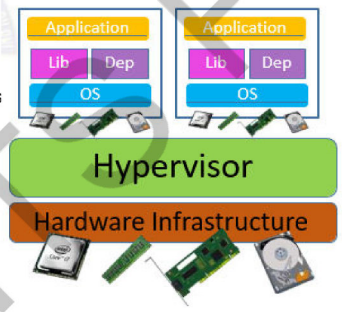
- Also known as "bare-metal"
- Interact directly with the hardware of the host machine
- Separate software tool is used to create and manipulate VM's
- Used in Data Centers

Pros

- Highly efficient
- Increased Security

Cons

- Need of separate management machine



Examples: Hyper-V, Xen, KVM, ESXi

Types of Hypervisor

Type 2

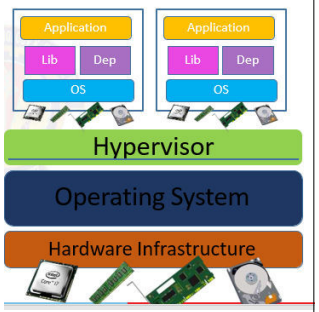
- Also known as "Hosted"
- Doesn't run directly on the underlying hardware
- Runs as an application in an OS
- Suitable for individual PC users

Pros

- Quick alternate OS

Cons

- Introduces latency
- Reduces performance



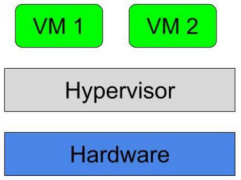
Examples: Virtual Box, QEMU, VMWare Player

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Virtual Machine

- A virtual computer system is known as a "virtual machine" (VM): a tightly isolated software container with an operating system and application inside.
- Each self-contained VM is completely independent.
- Putting multiple VMs on a single computer enables several operating systems and applications to run on just one physical server, or "host."



Properties of Virtual Machines

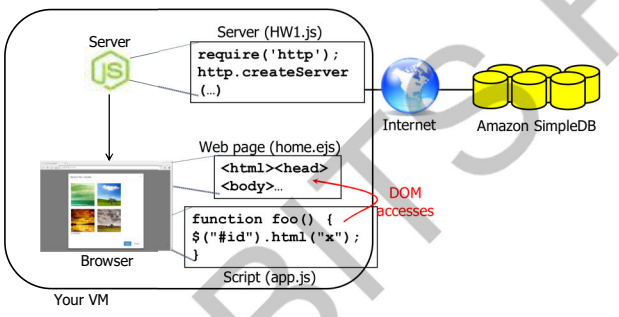
Partitioning
- ✓ Run multiple operating systems on one physical machine
- ✓ Divide system resources between virtual machines

Isolation
- ✓ Provide fault and security isolation at the hardware level
- ✓ Preserve performance with advanced resource controls

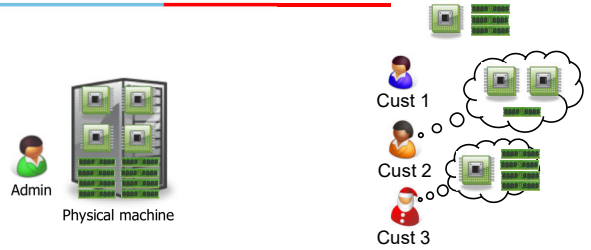
Encapsulation
- ✓ Save the entire state of a virtual machine to files
- ✓ Move and copy virtual machines as easily as moving and copying files

Hardware Independence
- ✓ Provision or migrate any virtual machine to any physical server

How does this work?



Use case Scenario for virtualization



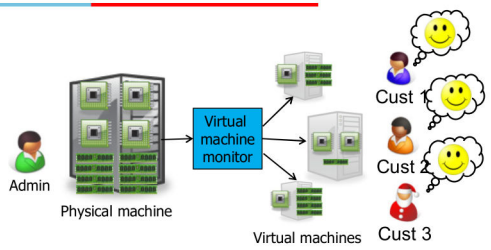
Suppose Admin has a machine with 4 CPUs and 8 GB of memory, and three customers:
- Cust 1 wants a machine with 1 CPU and 3GB of memory
- Cust 2 wants 2 CPUs and 1GB of memory
- Cust 3 wants 1 CPU and 4GB of memory

What should Admin do?

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

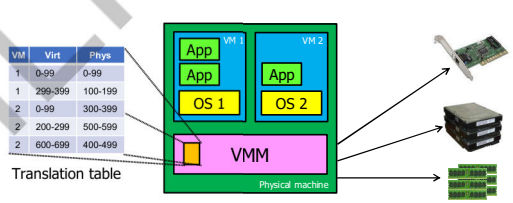
Resource allocation in virtualization



Admin can sell each customer a virtual machine (VM) with the requested resources

- From each customer's perspective, it appears as if they had a physical machine all by themselves (isolation)

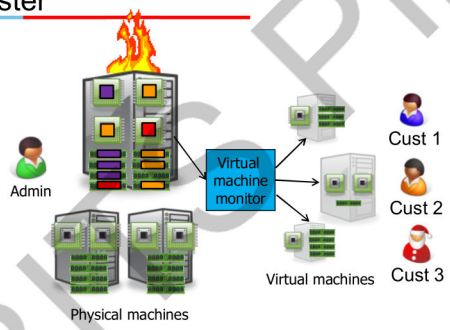
How does it work?



Resources (CPU, memory, ...) are virtualized

- VMM ("Hypervisor") has translation tables that map requests for virtual resources to physical resources
- Example: VM 1 accesses memory cell #323; VMM maps this to memory cell 123.
- For which resources does this (not) work?
- How do VMMs differ from OS kernels?

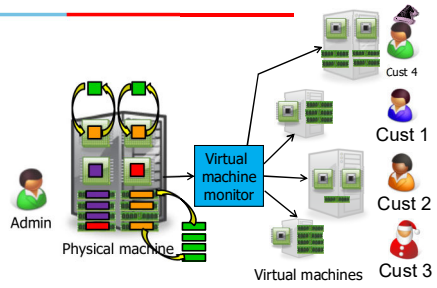
Benefit: Migration in case of disaster



What if the machine needs to be shut down?

- e.g., for maintenance, consolidation, ...
- Admin can migrate the VMs to different physical machines without any customers noticing

Benefit: Time sharing



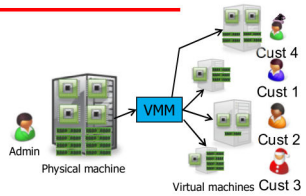
What if Admin gets another customer?

- Multiple VMs can time-share the existing resources
- Result: Admin has more virtual CPUs and virtual memory than physical resources (but not all can be active at the same time)

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Benefit and challenge: Isolation



Good: Cust 4 can't access Cust 3's data

Bad: What if the load suddenly increases?

- Example: Cust 4 VM shares CPUs with Cust 3's VM, and Cust 3 suddenly starts a large compute job
- Cust 4 performance may decrease as a result
- VMM can move Cust 4 's software to a different CPU, or migrate it to a different machine

Use Cases for Virtual Machines

1.IT: VMs can be used in IT to create virtual environments for testing new software and applications before deployment. This allows IT professionals to test the functionality of the software without risking the production environment. VMs are also used for disaster recovery and backup purposes.

2. Education: VMs can be used in education to provide students with access to a variety of software and applications without having to install them on individual computers. This allows Institutions to save money on licensing fees and hardware costs. Ex: BITS provides VMs for students in M.Tech programs to experiment.

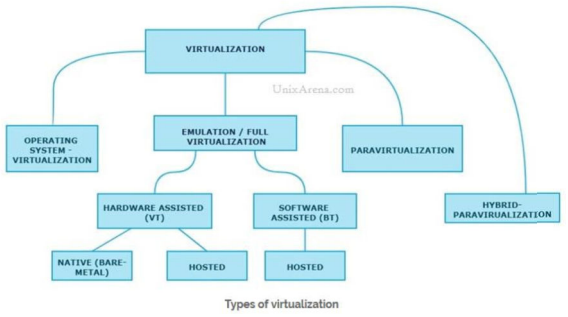
3. Finance: VMs can be used in finance to provide secure virtual environments for financial transactions and data storage. This allows financial institutions to reduce the risk of data breaches and improve the security of their systems.

4. Gaming: VMs can be used in gaming to provide players with access to a variety of games without having to install them on individual computers. This allows players to save money on game purchases and hardware costs.

Benefits of Virtualization

- Cost savings
- Improved resource utilization
- Increased scalability
- Improve Disaster recovery
- Better security
- Faster Provisioning of resources

Types of Virtualization



Types of virtualization

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Why Virtualization

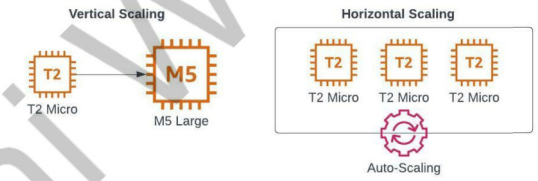
Load balancing- Applications can be moved from one physical machine to another by recreating the virtual machine on target physical HW

Consolidation - multiple VMs on the same physical HW - More efficient user of the HW and improved availability

Each app running in its own VM, cannot interfere with app running in other VMS - Multi-tenancy - a process can access only its own virtual address space and prevented from accessing the virtual address space of other processes:

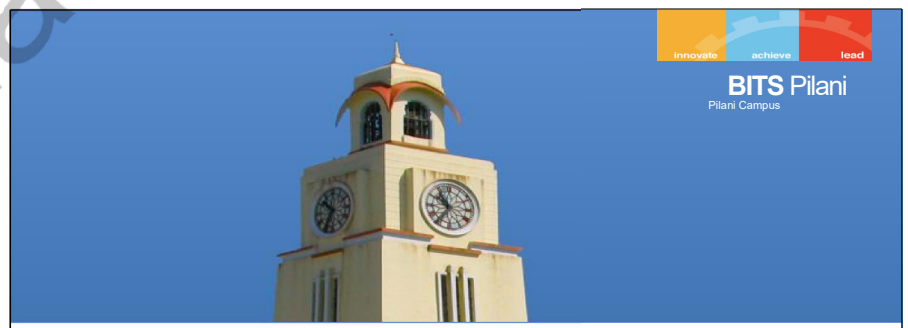
Vertical & Horizontal Scaling of VMs in Cloud Environment

- Vertical Scaling - Adding more power to an existing ec2 instance
  - Ex: Amazon RDS
- Horizontal Scaling - Adding more ec2 instances
  - Ex: MongoDB



Vertical & Horizontal Scaling - Comparison

Feature	Vertical Scaling	Horizontal Scaling
Data	Executed on a single node Consistent data	Partitioned and executed on multiple nodes; Inconsistent data
Downtime	While upgrading the machine	No downtime
Upper limit	Limited by machine specifications	Not limited by machine specifications
Load Balancing	Not required	Required
Failure	Single point of failure	Resilient to system failure



Containers and Docker

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

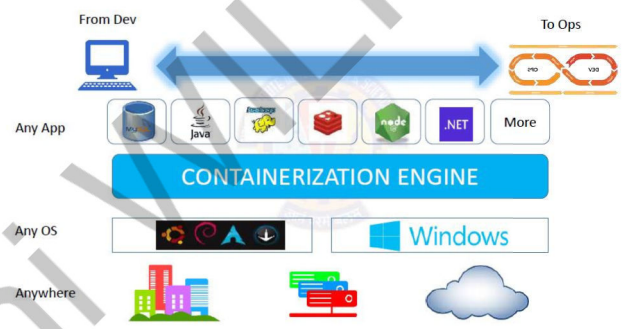
What are Containers

innovate achieve lead

- A software container is a standardized package of software
- Everything needed for the software to run is inside the container
- The software code, runtime, system tools, system libraries, and settings are all inside a single container
- **Container based deployments is favoured for Microservices**

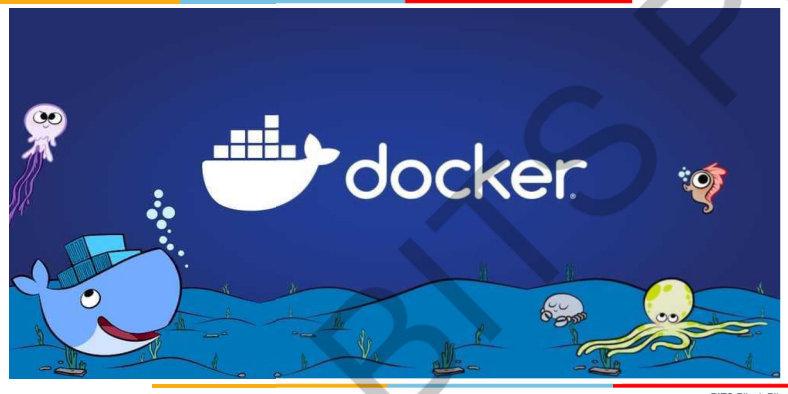
Containers – Build, Ship, Run Any App Anywhere

innovate achieve lead



Docker

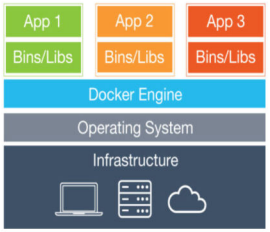
innovate achieve lead

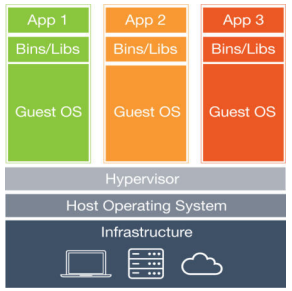


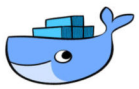
What is Docker?

innovate achieve lead

Packages an application into a container
Containers share the host OS kernel
Not a Virtual Machine (each VM has a separate OS)



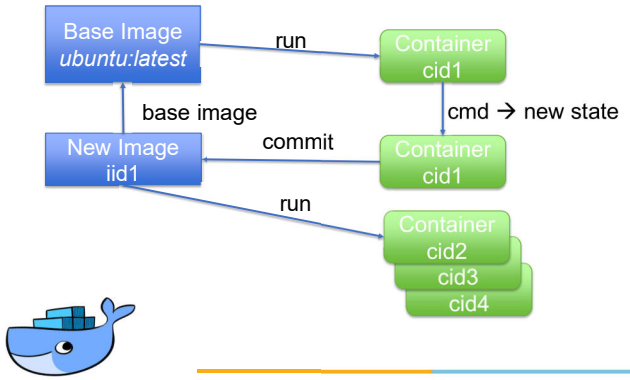




innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

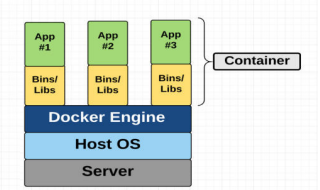
Image vs. Container



Containers made easy with

- Each container gets its own isolated user space to allow multiple containers to run on a single host machine.
- We can see that all the operating system level architecture is being shared across containers.
- The only parts that are created from scratch are the bins and libs.
- This is what makes containers so lightweight.

Source - https://docs.docker.com



Where does Docker come in?

- Docker is an open-source project based on Linux containers.
- It uses Linux Kernel features like namespaces and control groups to create containers on top of an operating system.
- Containers are far from new; Google has been using their own container technology for years.
- Others Linux container technologies include Solaris Zones, BSD jails, and LXC, which have been around for many years.

Source - https://docs.docker.com

Why is Docker all of a sudden gaining steam?

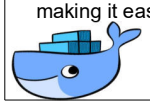
1. **Ease of use:** Docker has made it much easier for anyone — developers, systems admins, architects and others — to take advantage of containers in order to quickly build and test portable applications.
   - It allows anyone to package an application on their laptop, which in turn can run unmodified on any public cloud, private cloud, or even bare metal.
   - The mantra is: “build once, run anywhere.”
2. **Speed:** Docker containers are very lightweight and fast.
   - Since containers are just sandboxed environments running on the kernel, they take up fewer resources.
   - You can create and run a Docker container in seconds, compared to VMs which might take longer because they have to boot up a full virtual operating system every time.

innovate achieve lead

Why is Docker all of a sudden gaining steam?

3. **Docker Hub:** Docker users also benefit from the increasingly rich ecosystem of Docker Hub, which you can think of as an "app store for Docker images."
- Docker Hub has tens of thousands of public images created by the community that are readily available for use.
- It's incredibly easy to search for images that meet your needs, ready to pull down and use with little-to-no modification.

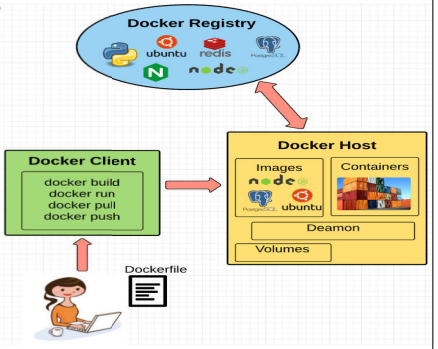
4. **Modularity and Scalability:** Docker makes it easy to break out your application's functionality into individual containers.
- For example, you might have your Postgres database running in one container and your Redis server in another while your Node.js app is in another.
- With Docker, it's become easier to link these containers together to create your application, making it easy to scale or update components independently in the future.



Fundamental Docker Concepts

Now that we've got the big picture in place,

let's go through the fundamental parts of Docker piece by piece:



Docker Engine

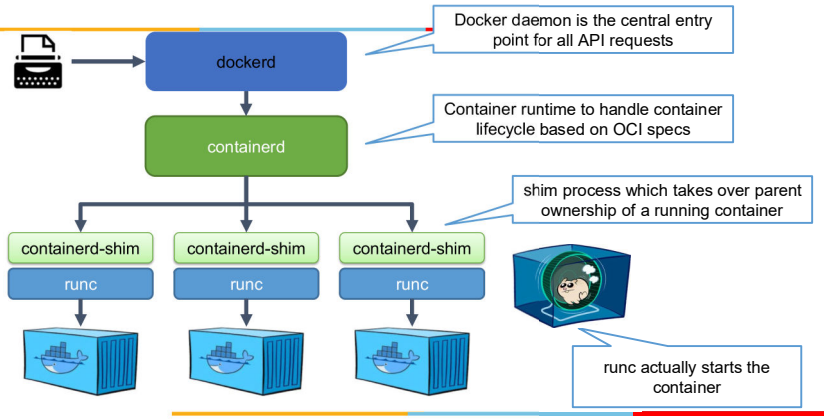
Docker engine is the layer on which Docker runs.

It's a lightweight runtime and tooling that manages containers, images, builds, and more.

It runs natively on Linux systems and is made up of:

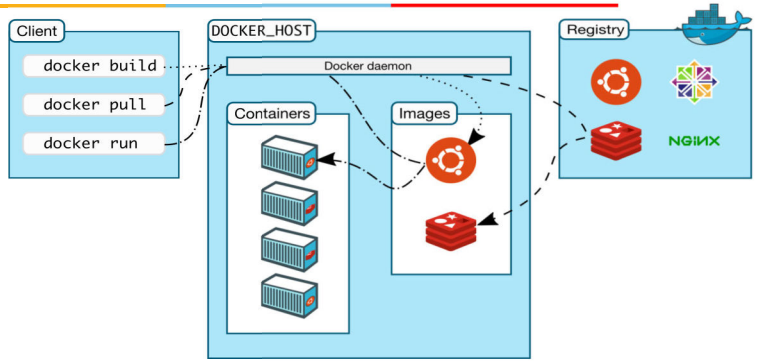
1. A Docker Daemon that runs in the host computer.
2. A Docker Client that then communicates with the Docker Daemon to execute commands.
3. A REST API for interacting with the Docker Daemon remotely.

A look into the docker engine



BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Docker eco-system



https://docs.docker.com/engine/docker-overview/

Docker eco-system

Docker Client

The Docker Client is what you, as the end-user of Docker, communicate with.

Think of it as the UI for Docker. For example, when you do…

you are communicating to the Docker Client, which then communicates your instructions to the Docker Daemon.

Docker Daemon

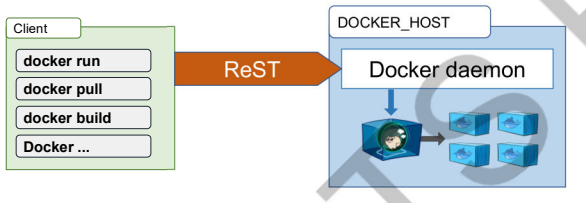
The Docker daemon is what actually executes commands sent to the Docker Client — like building, running, and distributing your containers.

The Docker Daemon runs on the host machine, but as a user, you never communicate directly with the Daemon.

The Docker Client can run on the host machine as well, but it's not required to.

It can run on a different machine and communicate with the Docker Daemon that's running on the host machine.

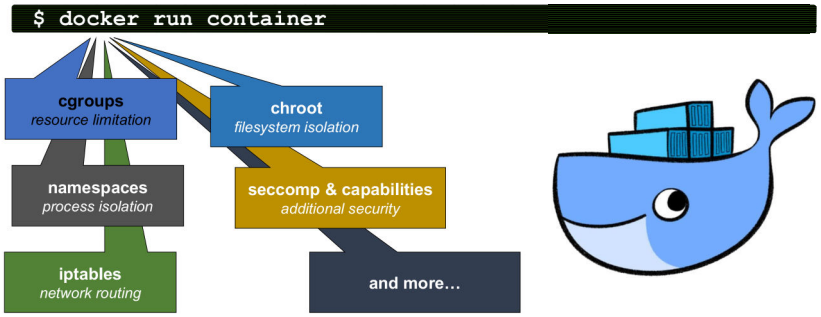
Docker's client/server architecture



- Docker client instructs Docker daemon what to do – the real hard work is done by the Docker daemon

Source - https://docs.docker.com

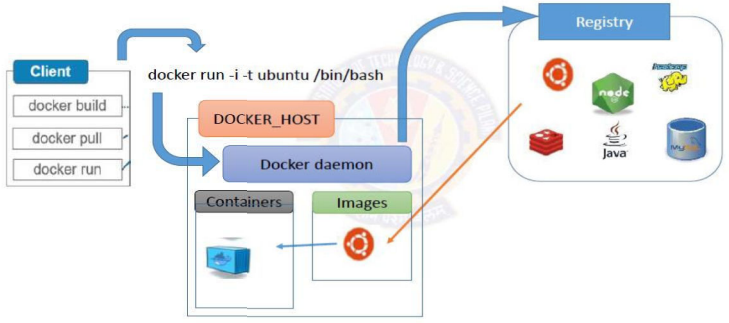
Let's start our first container… the easy way!



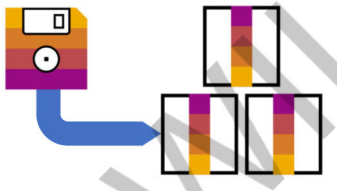
Source - https://docs.docker.com

innovate achieve lead

Running container…



Images



Images are filesystem snapshots that function as the root filesystem of a container at startup + some metadata.

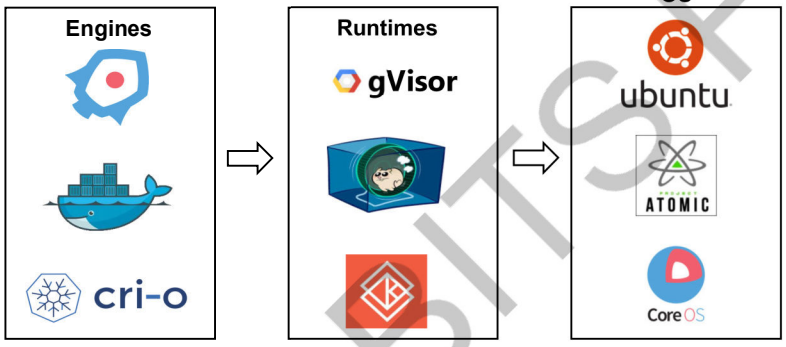
They can be

- pulled from a registry
- created from a Dockerfile or by committing changes.

Images consist of several layers that are stacked upon each other.

```
$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED
SIZE
test_nginx          latest              ee56ce14b382        3 days
ago         108MB
<none>              <none>              2af6607c94b9        13 days
ago         108MB
debian              latest              6d83de432e98        4 weeks
```


Open Container Initiative



Docker Hub

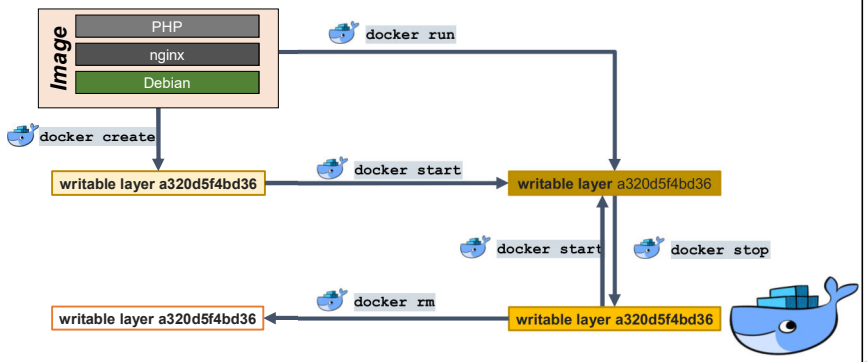
Docker's analog of GitHub for images instead of code
Find images (ubuntu, redis, mysql, mongo, node, postgres, etc)
Push / Pull your own images (public)
>1 private image is paid

Also, Docker Trusted Registry is available if you must maintain control over your own code.

innovate achieve lead

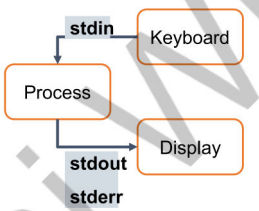
BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Lifecycle of a container



Detached, interactive, tty, tt-what?

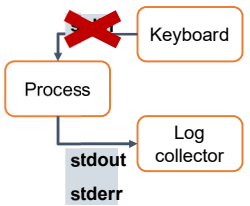
docker run –i –t ubuntu /bin/bash



Running an interactive session requires -i & -t :

-t allocates a pseudo-tty, -i keeps stdin open.

docker run –d ubuntu /bin/httpd



Running a detached session keeps process 1 alive without stdin open.

Information about containers

Available / present containers

- the command **docker container list** gives a list of all running containers on a host (use –a to see terminated containers as well)

```
$ docker ps -a

CONTAINER ID    IMAGE             COMMAND                  CREATED             STATUS
PORTS                    NAMES
dded97d9a7d6    docker/whalesay   "cowsay 'Rick Astley'"   8 minutes ago       Exited (0) 8 minutes
ago                      ecstatic_goldstine
b69038e9ff35    busybox           "sh"                     About an hour ago   Created
admiring_ritchie
e7a7aeca8c41    test_nginx        "nginx -g 'daemon ..."   4 days ago          Exited (1) 24 hours
ago                      pensive_elion
```

image

unique ID for each container (truncated)

program that runs within the container

creation time

current status and exit code

forwarded network ports

friendly name

Executing commands in a container

It is possible to run additional commands in a container

- Useful for debugging
- Command must be present in the container
- Interactive/detached mode – also applies here

```
$ docker container exec -i -t <container ID or name> <command>
```

```
$ docker exec -i -t 494b7b8c9f39 /bin/sh
# ls -al /usr/sbin/nginx -rwxr-xr-x 1 root root 1253840  Apr
24 14:28 /usr/sbin/nginx
# exit
```


innovate achieve lead

Getting logs from a container

Docker collects logs from programs inside containers

- stdout and stderr of the main process get collected
- logs from programs can be redirected to /dev/stdout and /dev/stderr
- logs still available even after a container terminated

```
$ docker logs <container ID or name>
```

```
$ docker container logs 494b7b8c9f39
10.19.91.230 - - [01/Apr/2024:14:48:12 +0000] "GET / HTTP/1.1" 200 230 "-" "Mozilla/5.0
(Windows NT 10.0; Win64; x64; rv:57.0) Gecko/20100101 Firefox/57.0" "-"
10.19.91.230 - - [01/Apr/2024:14:48:12 +0000] "GET /it_works.jpg HTTP/1.1" 200 25676
"http://pvxka22:32780/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:57.0) Gecko/20100101
Firefox/57.0" "-"
2017/12/01 14:48:12 [error] 8#8: *1 open() "/srv/www/htdocs/favicon.ico" failed (2: No such
file or directory), client: 10.19.91.230, server: localhost, request: "GET /favicon.ico
HTTP/1.1", host: "pvxka22:32780"
10.19.91.230 - - [01/Apr/2024:14:48:12 +0000] "GET /favicon.ico HTTP/1.1" 404 169 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:57.0) Gecko/20100101 Firefox/57.0" "-"
```


Stopping a container

A container can be forcibly stopped

- necessary for containers with daemon processes
- friendly termination with SIGTERM, after timeout (default 10 seconds) with SIGKILL *

```
$ docker stop <container ID or name>
```

```
$ docker container list
CONTAINER ID    IMAGE       COMMAND          CREATED      STATUS       PORTS
NAMES
494b7b8c9f39    mynginx1    "nginx -g ..."   7 days ago   Up 24 hours  0.0.0.0:32768-
>812/tcp   happy_ride

$ docker container stop -t 5 happy_ride
494b7b8c9f39

$ docker container list
CONTAINER ID    IMAGE       COMMAND          CREATED      STATUS       PORTS
```

* you can specify wait time between SIGTERM and SIGKILL with the "-t" option

Removing a container

Finally, a container can and should be removed

- all data in the container that was not part of the image is lost
- when a container is removed, it is gone
- removing an active container must be forced and will kill it

```
$ docker rm <container ID or name>
```

```
$ docker container list -a
CONTAINER ID   IMAGE              COMMAND           CREATED             STATUS
PORTS          NAMES
dded97d9a7d6   docker/whalesay    "cowsay 'Ri..."   About an hour ago   Exited (0)
happy_pony
9682e4fba8b7   busybox            "sh"              25 hours ago        Up 25 hours
elated_saha

$ docker container rm 9682e4fba8b7 dded97d9a7d6
dded97d9a7d6
Error response from daemon: You cannot remove a running container…

$ docker container rm 9682e4fba8b7 --force
9682e4fba8b7

$ docker container list -a
CONTAINER ID   IMAGE              COMMAND           CREATED             STATUS
```

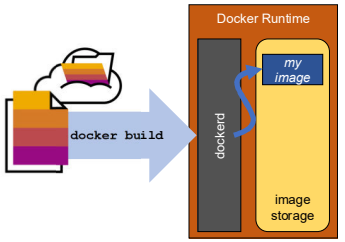

Creating images with Dockerfile(s)

Creating a Docker image involves two things

- A Dockerfile containing directives on how the image is meant to be built
- A build-context (or PATH) which is a directory on your hard disk
  - every file to be added to the image must be inside the build-context

Image is built by the Docker daemon

- the whole build-context is sent to the daemon
- Dockerfile is processed
- Image stored in the local image store



"**Warning: Do not use your root directory, /, as the PATH as it causes the build to transfer the entire contents of your hard drive to the Docker daemon.**"
https://docs.docker.com/engine/reference/builder/#usage